PUBLIC PORTFOLIO TECHNICAL CASE STUDY

# SmartCH4 Friendly

Industrial biogas monitoring, feed planning, optimization, and methane prediction

**Full-stack application | secure identity | sensor data | decision-service integration | delivery engineering**

*Commercial source code and private operational details are not included.*

01 | CONTEXT

# Executive summary

**What I delivered:** A secure web platform that turns plant settings, sensor telemetry, laboratory feedstock analyses, and recent supply history into an operator-guided optimization and prediction workflow.

SmartCH4 Friendly was built for the operational reality of a biogas unit: measurements arrive at different frequencies, laboratory values need human review, feedstock properties change, and mathematical or machine-learning services must be presented as decisions an operator can understand and retain.

## Project at a glance

| Area | Public description |
|---|---|
| **Business goal** | Connect biogas operations data to repeatable monitoring, planning, optimization, and prediction decisions. |
| **My role** | Full-stack implementation, integration, data design, security integration, containerization, and deployment preparation. |
| **Primary users** | Authenticated plant operators working in a facility-specific data context. |
| **Source availability** | Private because the implementation was produced as paid client work. |
| **Backend** | Java 21, Spring Boot |
| **Frontend** | React, TypeScript, Ant Design |
| **Database** | MySQL |

02 | ARCHITECTURE

# System architecture

The application is an orchestration platform. The browser owns operator interaction; Keycloak owns identity; Spring Boot owns business rules, facility scoping, data consistency, and decision-service coordination; MySQL owns operational history; the solver and model remain independently deployable.

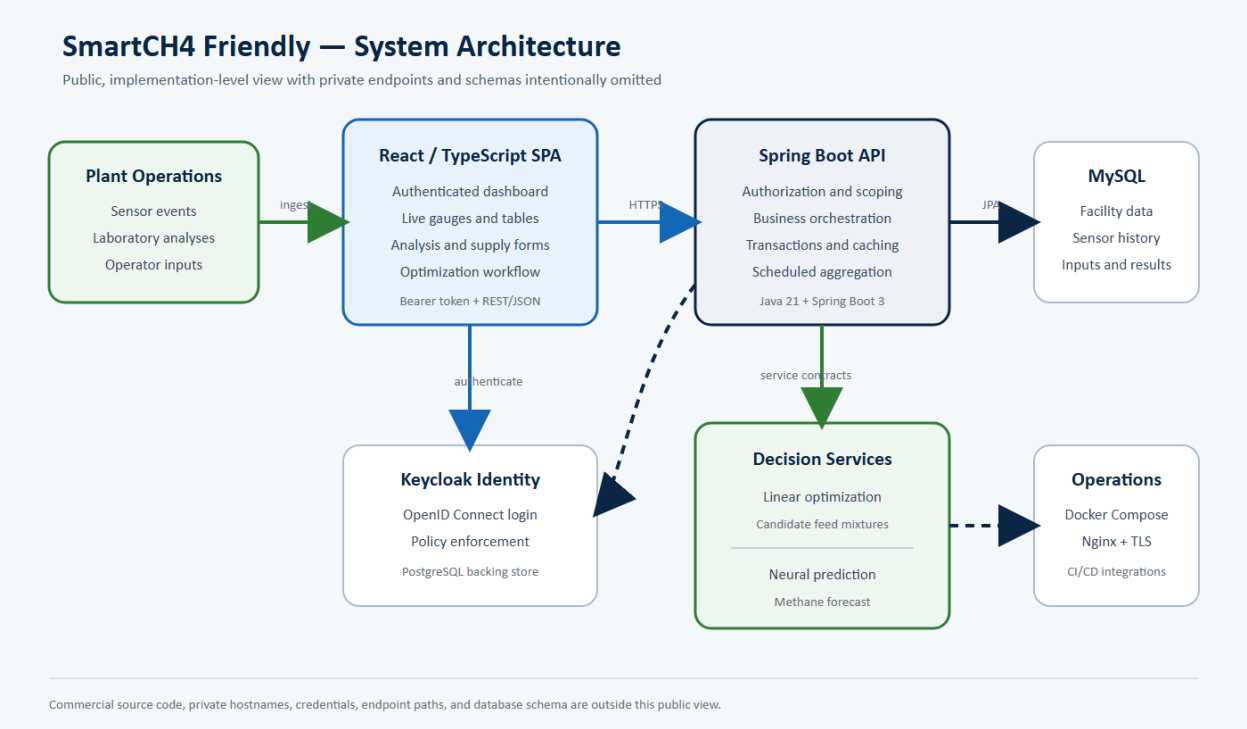


*Figure 1. Public architecture view; private endpoints and database schema are intentionally omitted.*

## Key boundaries

- The React client communicates only through authenticated HTTPS API calls.
- The backend uses stateless bearer-token enforcement and resolves a facility context before reading or writing operational data.
- Optimization and prediction are invoked through dedicated HTTP clients rather than embedded in the application process.
- MySQL stores the application domain; Keycloak uses a separate PostgreSQL identity store; and Nginx routes identity, API, and frontend traffic through the public TLS entry point.

03 | PRODUCT WORKFLOW

## From plant data to a saved decision

The product workflow is staged so that operators can inspect and correct the evidence used by the decision services instead of receiving an unexplained number.

1. Authenticate and resolve the user account to its facility record.
2. Maintain operating parameters such as capacity, daily feed, methane target, reactor volume, and retention time.
3. Ingest current sensor events and review the latest process values on the dashboard.
4. Aggregate a selected day into a model-ready sensor record, with manual correction available when field or laboratory data requires reconciliation.
5. Maintain substrate analyses covering availability, economics, composition, and methane potential.
6. Record the facility's recent daily supply mixtures and calculate their nutrient totals.
7. Select eligible substrates, request optimized mixture candidates, review the alternatives, and persist the preferred solution.
8. Send recent operating history plus the chosen future composition to the prediction service, then retain the forecast with the result.
9. Compare same-day runs and save the preferred daily result for later use.

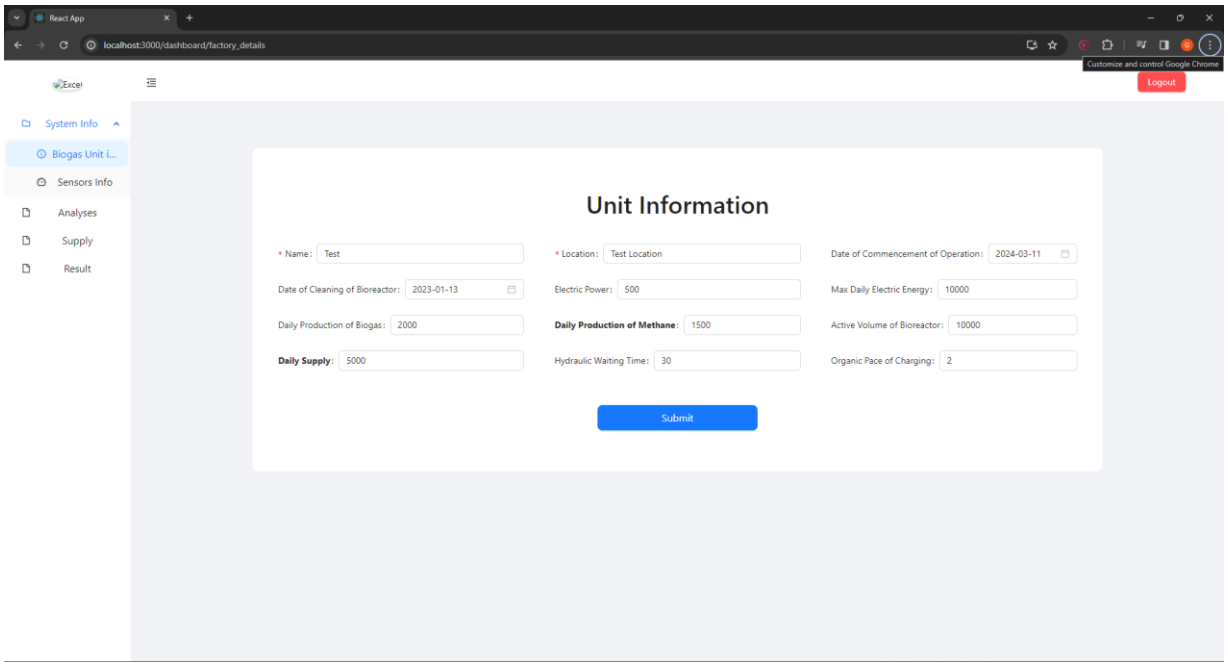


*Figure 2. Facility configuration screen with representative test data.*

04 | SENSOR ENGINEERING

# Sensor ingestion and daily processing

The sensor layer supports both single-reading and batch ingestion. Each reading is associated with a facility and timestamp so the application can present the latest state, calculate a selected day's values, and retain the history required by prediction.

## Processing model

- Temperature, pH, redox, and ammonium are treated as sampled measurements; the daily record uses rounded arithmetic averages.
- Biogas and methane are treated as pulse/count-derived measurements; daily quantities are calculated with environment-specific calibration factors rather than exposing those factors publicly.
- The combined daily record also accepts laboratory-derived volatile fatty acid and ammonia values needed by the prediction model.
- A scheduled job prepares the preceding day's aggregate for every known facility, while the user interface can retrieve or edit a selected date.
- Read-heavy current and daily views use caching, and mutations explicitly invalidate the relevant cache entries.
- Weekly retention tasks remove older raw sensor records while preserving the higher-level decision records.

**Data-quality decision:** Operator correction is intentional. Industrial telemetry may be delayed, reset, recalibrated, or supplemented by laboratory results, so the application preserves a controlled way to reconcile the daily feature record before prediction.

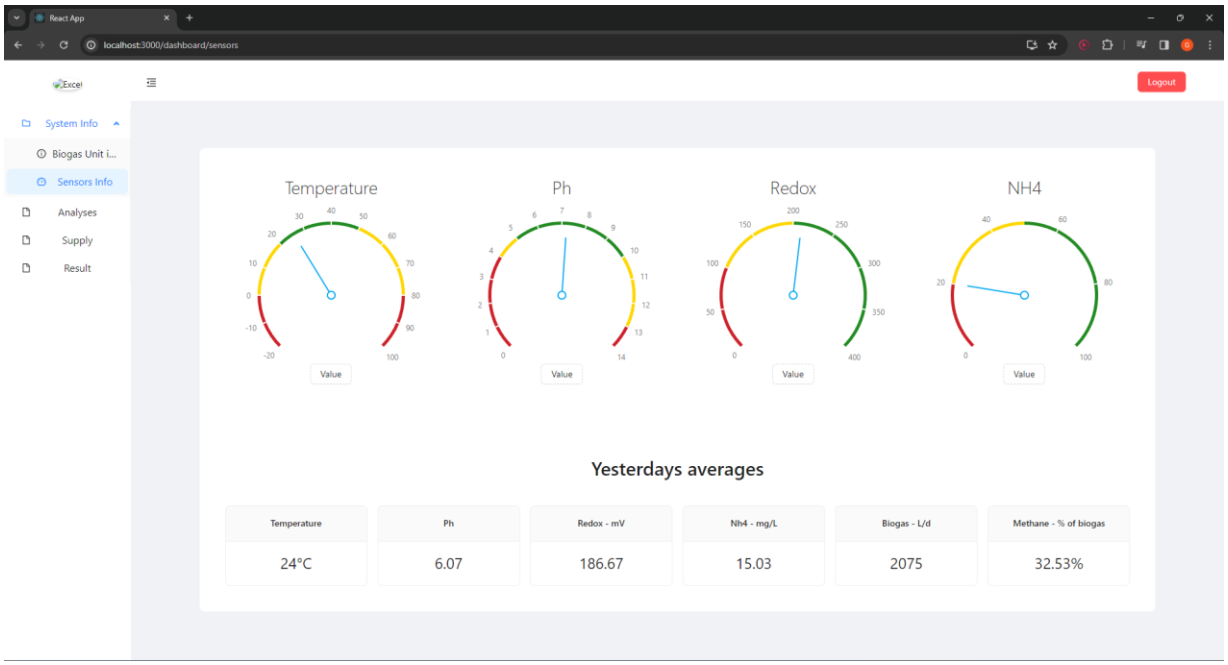


*Figure 3. Live process gauges and daily aggregate cards using representative data.*

05 | DOMAIN MODELING

# Feedstock knowledge and weekly supply

The substrate analysis catalog bridges laboratory knowledge and operational planning. Each record supplies the constraints and coefficients needed to judge availability, cost, composition, and methane potential.

## Analysis dimensions

| Dimension | Representative information used by the workflow |
|---|---|
| **Identity** | Human-readable name, type, source, analysis date, and owning facility. |
| **Availability** | Quantity available at the source and already present at the unit. |
| **Economics** | Distance, purchase cost, and modeled feedstock cost. |
| **Composition** | Solids, carbohydrates, proteins, fat, ammonium, and volatile fatty acids. |
| **Gas potential** | Biogas and methane potential used by optimization. |

## Supply aggregation

A daily supply entry combines analyses and quantities. The service calculates weighted carbohydrate, protein, and fat totals, stores the composition transactionally, and exposes the latest seven-day window as prediction context.

- Composite association records prevent duplicate substrate entries inside the same supply or result.
- Create and update operations load the referenced analyses in batches and rebuild totals from persisted domain values.

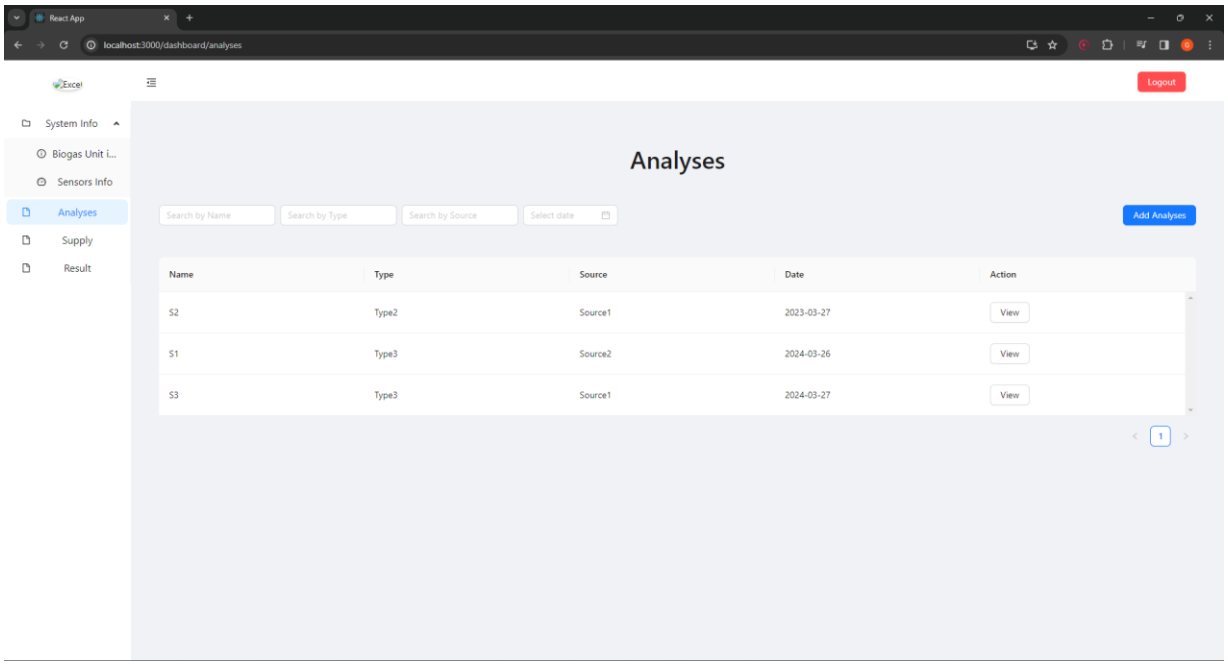


*Figure 4. Searchable substrate analysis catalog with representative records.*

06 | DECISION SERVICES

# Optimization and prediction orchestration

## Linear optimization

When the operator selects candidate substrates, the backend enriches those choices with the facility's daily methane target and the analysis values required by the external solver. It requests multiple solutions, keeps the solver responses marked feasible/optimal, rounds display totals consistently, maps solver names back to internal analysis records, and returns operator-ready alternatives.

- Input: eligible substrates, combined availability, nutrient composition, methane potential, cost, distance, and facility target.
- Output: candidate mixture components and totals for quantity, cost, methane, carbohydrates, protein, and fat.
- User control: the operator reviews candidate details and explicitly chooses which result becomes a persisted domain record.

## Neural prediction

Prediction is requested after a result exists, which gives the platform a stable identifier and an auditable record even when the external model is slow or unavailable. The request builder joins recent supply composition with per-day process features and the chosen future composition.

| Model segment | Feature groups | Purpose |
|---|---|---|
| **Past sequence** | Seven days of feed nutrients, volatile fatty acids, ammonia, biogas, and methane. | Describe the recent operating state. |
| **Future vector** | Carbohydrates, protein, and fat in the chosen optimized mixture. | Describe the proposed next input. |
| **Response** | One or more methane predictions, reduced to a rounded stored value. | Compare model forecast with the optimization estimate. |

## Resilience at the user boundary

- The result remains available when the prediction service cannot return a value.
- The client shows loading and retry states, gives a clear unavailable message, allows another prediction request, and preserves same-day result comparison.

07 | BACKEND

# Backend engineering

The Java service follows a conventional layered Spring design, but the important work sits in the service layer: constructing compound aggregates, enforcing facility context, translating between API and persistence models, coordinating external services, and keeping derived totals consistent.

| Layer | Responsibility in this project |
|---|---|
| **Controllers** | Expose the application domains as REST operations and translate request parameters into service calls. |
| **DTOs** | Separate external request/response shapes from JPA entities and the solver/model contracts. |
| **Services** | Apply business rules, aggregate values, coordinate persistence, resolve identity context, and invoke decision services. |
| **Repositories** | Provide facility- and date-scoped queries, latest-record lookup, uniqueness checks, and retention deletes. |
| **Entities** | Model facilities, sensor records, analyses, daily supply, generated results, saved results, and composition links. |
| **Configuration** | Wire mapping, CORS, Keycloak clients, policy enforcement, cache behavior, and declarative HTTP clients. |

## Consistency mechanisms

- Transactional service methods store a parent record and its composition links as one unit of work.
- Database uniqueness constraints are translated into domain-specific exceptions rather than leaking raw persistence errors.
- Batch lookup of referenced analyses avoids repeated per-row database reads during compound writes.
- Caching covers common facility, analysis, supply, sensor, and saved-result reads, with explicit invalidation after mutation.
- External response objects are adapted into domain-owned result records rather than persisted as opaque third-party payloads.

08 | FRONTEND

# Frontend engineering and operator experience

The React client is organized around the operator's sequence of work rather than the backend's entity list. A persistent dashboard shell exposes facility information, sensors, analyses, supply, results, and saved decisions; each module uses focused tables, forms, and modals.

## Interaction design

- A collapsible navigation shell keeps the six operational modules available without leaving the authenticated workspace.
- The sensor screen refreshes current values automatically and combines gauges, thresholds, date selection, daily values, and correction inputs.
- Analysis and supply screens use searchable or drill-down tables with create, edit, and delete modals.
- The result screen is a four-step workflow: choose analyses, choose a candidate solution, request/review prediction, and compare final results.
- Saved results preserve a compact summary table with modal access to the selected substrate composition.
- Ant Design validation, confirmation dialogs, loading messages, success/error feedback, and disabled states make asynchronous operations visible.

## Authentication behavior

The application initializes with login required. An Axios request interceptor refreshes the Keycloak token when necessary and adds the bearer token to every API request. Logout clears the local facility context and ends the Keycloak session.

## State and data flow

- Component state controls filters, modal selection, result steps, retry counts, and responsive widths.
- Facility identity is cached client-side only as a convenience; the backend still resolves and applies the owning facility to domain operations.
- Date handling normalizes form values before transmission so daily records align with backend date types.

09 | DATA AND SECURITY

# Domain model, identity, and facility scoping

## High-level domain relationships

| Domain | Relationship and purpose |
|---|---|
| **Facility** | Root operating context for plant settings and all facility-owned records. |
| **Sensor records** | Timestamped raw measurements plus one combined daily feature record. |
| **Analysis** | Facility-owned feedstock characteristics used by supply and optimization. |
| **Supply** | A dated mixture of analyses and quantities, with derived nutrient totals. |
| **Result** | A persisted chosen optimization candidate, its components, totals, and optional prediction. |
| **Saved result** | A daily snapshot copied from a result so the operator can retain a preferred decision. |

## Security flow

10. Keycloak provides OpenID Connect login and issues the browser's access token.
11. The frontend refreshes and attaches the token to API calls.
12. Spring Security runs statelessly and applies a Keycloak policy-enforcement filter.
13. The token subject is resolved to a Keycloak username, which establishes or locates the matching facility record.
14. Facility-aware queries and service methods use the facility identifier when reading analyses, supplies, sensors, results, or saved data.

## Public-data discipline

This public package omits identity exports, credentials, internal hostnames, private IP addresses, database field-level diagrams, endpoint paths, certificates, service secrets, and configuration files. The selected screenshots contain representative test data only.

10 | DELIVERY

# Deployment and operations

## Container topology

- A Java 21 container runs the packaged Spring Boot application.
- A Node-based container builds/runs the React client in the legacy snapshot.
- MySQL stores the operational domain, while Keycloak and PostgreSQL handle identity.
- Nginx terminates TLS, redirects clear-text traffic, and reverse-proxies the frontend, API, and identity routes.
- Docker Compose joins the services through named networks and persistent volumes.

## CI/CD preparation

The repository contains deployment preparation for a Jenkins controller configured as code, role-based Jenkins access, ephemeral Docker agents, Maven and JDK tool installation, source-control credentials, SonarQube integration, and an artifact registry. Supporting Compose material provisions the identity, quality, and artifact services behind Nginx.

| Pipeline phase | Documented activities |
|---|---|
| **Build and unit check** | Compile/package with Maven, run unit tests, and publish build artifacts. |
| **Quality and image** | Run static analysis, build the container, and perform image scanning. |
| **Registry and development** | Push versioned images and deploy to a development environment. |
| **Validation environments** | Run smoke, integration, manual, and performance gates across staged environments. |
| **Release** | Promote through client testing to production and disaster-recovery environments. |

11 | QUALITY

# Engineering quality and legacy status

## What the implementation does well

- Keeps identity, operational persistence, mathematical optimization, and neural prediction as separate system responsibilities.
- Models compound mixtures explicitly instead of flattening substrate composition into unstructured text.
- Uses transactions for multi-entity writes and domain exceptions for common conflict/not-found cases.
- Preserves operator review and correction where sensor and laboratory data can diverge.
- Treats a prediction as an optional external result rather than allowing model failure to erase the chosen optimization record.
- Includes public/private deployment boundaries and a containerized route to repeatable environments.

12 | CONTRIBUTION

# Contribution summary and portfolio framing

The strongest story in this project is not any single framework. It is the end-to-end translation of a domain workflow into a secure application that coordinates messy plant data, relational records, human review, deterministic optimization, and probabilistic prediction.

## My contribution, stated precisely

- Designed and implemented the facility, sensor, analysis, supply, result, and saved-result application modules.
- Built the Spring Boot REST and service layers, JPA relationships, DTO boundaries, transactional writes, caching, scheduled processing, and exception handling.
- Built the React/TypeScript operations dashboard, forms, tables, gauges, modal editors, and step-based decision flow.
- Integrated Keycloak login, token refresh, backend policy enforcement, and identity-to-facility resolution.
- Integrated independent linear-optimization and neural-prediction services and translated their contracts into operator-facing decisions.
- Containerized the web application and prepared/adapted Nginx, Keycloak, database, Jenkins, SonarQube, and artifact-registry deployment assets.
- Worked from evolving domain requirements, including sensor calibration behavior, daily aggregation, feedstock composition, and model-input history.

## Resume-ready bullets

| Focus | Suggested bullet |
|---|---|
| **Full-stack** | Built an authenticated industrial decision-support platform joining biogas sensor monitoring, laboratory feedstock data, weekly supply planning, optimization, and methane prediction. |
| **Backend** | Implemented a Java 21/Spring Boot API with transactional JPA persistence, per-facility data scoping, scheduled aggregation, caching, and two external decision-service integrations. |
| **Frontend** | Delivered a React/TypeScript operations dashboard with live gauges, searchable records, multi-step solution selection, prediction feedback, and saved-result workflows. |
| **Delivery** | Containerized the stack and prepared HTTPS, Keycloak, database, Jenkins, SonarQube, and artifact-registry deployment assets. |

13 | DISCLOSURE

# Public disclosure boundary

The goal of this case study is to make the engineering work visible without transferring commercial implementation details. The public folder is intentionally self-contained so it can be published without the private application tree.

| Included publicly | Excluded from publication |
|---|---|
| High-level problem, architecture, workflows, and technology stack | Application source code and compiled artifacts |
| Representative screenshots using test data | Client data, user records, identity exports, and certificates |
| Implementation-scale counts and engineering decisions | Exact endpoint paths, service addresses, IPs, and database schema |

## Final note

This document describes the completed implementation while keeping the paid project's code and operational configuration private.

**SmartCH4 Friendly** | Public technical case study